Practical 10: Currency Converter Using EJB

1. Aim

To develop a Currency Converter application using Enterprise JavaBeans (EJB) to convert US Dollars (USD) into Indian Rupees (INR).

2. First, understand your teacher's project

Your ZIP contains the following project structure:

```
CurrencyConverter
│
├── CurrencyConverter-ejb
│   └── Source Packages
│       └── com.demo.ejb
│           ├── CurrencyConverterBean.java
│           ├── CurrencyConverterLocal.java
│           └── CurrencyConverterBeanLocal.java
│
└── CurrencyConverter-war
    ├── Source Packages
    │   └── com.demo.web
    │       └── CurrencyServlet.java
    │
    └── Web Pages
        └── index.html
```

Your teacher has divided the application into two projects:

- **EJB project:** Contains the business logic for currency conversion.
- **Web project:** Contains the servlet that calls the EJB and displays the result.

We will keep this structure.

---

3. File 1: CurrencyConverterLocal.java

Location

Open:

CurrencyConverter → CurrencyConverter-ejb → Source Packages → com.demo.ejb → CurrencyConverterLocal.java

Code from your ZIP

```
package com.demo.ejb;

import jakarta.ejb.Local;

@Local
public interface CurrencyConverterLocal {

   double convertToINR(double usd);

}
```

Explanation

- package com.demo.ejb; specifies the package.
- import jakarta.ejb.Local; imports the annotation for a local EJB interface.
- @Local declares the interface as a local business interface.

- convertToINR(double usd) defines the method that converts USD into INR.
- The method returns a double value.

Do we need to correct this code?
No. This interface is correct for the application.

---

4. File 2: CurrencyConverterBean.java
Location
Open:
CurrencyConverter → CurrencyConverter-ejb → Source Packages → com.demo.ejb → CurrencyConverterBean.java
Code from your ZIP

```
package com.demo.ejb;

import jakarta.ejb.Stateless;

@Stateless
public class CurrencyConverterBean implements CurrencyConverterLocal {

  @Override
  public double convertToINR(double usd) {
    return usd * 86.5;
  }
}
```

Explanation
1. @Stateless
Declares the class as a stateless session bean managed by the application server.
2. implements CurrencyConverterLocal
The bean implements the local interface created in the previous step.
3. @Override
Indicates that the method implements a method declared in the interface.
4. convertToINR(double usd)
Accepts a USD amount and converts it into INR.
5. return usd * 86.5;
Multiplies the USD amount by the example conversion rate of ₹86.50 per USD.
For example:
USD = 10

INR = 10 × 86.5

INR = 865
Do we need to correct this code?
The code is valid. The conversion rate is fixed at 86.5, so this is a demonstration rather than a live currency converter. The result depends on the rate specified in the program.
What about CurrencyConverterBeanLocal.java?
Your ZIP also contains a file named CurrencyConverterBeanLocal.java, but it is an empty interface.

The bean implements CurrencyConverterLocal, not CurrencyConverterBeanLocal. Therefore, the empty interface is not needed for this implementation. You can leave it in the project without using it.

---

5. File 3: CurrencyServlet.java

Location

Open:

CurrencyConverter → CurrencyConverter-war → Source Packages → com.demo.web → CurrencyServlet.java

Your teacher's servlet uses EJB injection to call the conversion method.

Corrected code

The original servlet has no compilation error, but it always converts exactly **1 USD**. We can improve it to accept an amount from the user while preserving your teacher's EJB structure.

Replace the contents of CurrencyServlet.java with:

```
package com.demo.web;

import com.demo.ejb.CurrencyConverterLocal;
import jakarta.ejb.EJB;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/CurrencyConverterLocal")
public class CurrencyServlet extends HttpServlet {

  @EJB
  private CurrencyConverterLocal converter;

  @Override
  protected void doGet(HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {

    response.setContentType("text/html;charset=UTF-8");

    PrintWriter out = response.getWriter();

    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Currency Converter</title>");
    out.println("</head>");
```

```
        out.println("<body>");

        out.println("<h2>Currency Converter using EJB</h2>");
        out.println("<form method='post' action='CurrencyConverterLocal'>");
        out.println("Enter USD Amount: ");
        out.println("<input type='number' name='usd' min='0.01' step='any' required>");
        out.println("<input type='submit' value='Convert'>");
        out.println("</form>");

        out.println("</body>");
        out.println("</html>");
    }

    @Override
    protected void doPost(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        PrintWriter out = response.getWriter();

        try {
            double usd = Double.parseDouble(request.getParameter("usd"));

            if (!Double.isFinite(usd) || usd <= 0) {
                out.println("<h3>Please enter a valid positive amount.</h3>");
                return;
            }

            double inr = converter.convertToINR(usd);

            out.println("<html><body>");
            out.println("<h2>Currency Converter using EJB</h2>");
            out.println("<p><b>USD Amount:</b> " + usd + "</p>");
            out.println("<p><b>Converted INR:</b> "
                    + String.format("%.2f", inr) + "</p>");
            out.println("<a href='CurrencyConverterLocal'>Convert Again</a>");
            out.println("</body></html>");

        } catch (NumberFormatException e) {
            out.println("<h3>Please enter a valid number.</h3>");
        }
    }
}
```

What was corrected?

Your teacher's original code:

- Uses doGet() to calculate the conversion.
- Hardcodes the USD amount to 1.
- Displays the result immediately without asking the user for an amount.

The corrected version:

- Uses doGet() to display a form.
- Uses doPost() to receive the amount.
- Calls the same EJB business method.
- Checks that the entered amount is a valid positive number.

The central EJB concept remains unchanged:

@EJB
private CurrencyConverterLocal converter;

This annotation allows the application server to inject the EJB reference into the servlet.

---

6. File 4: index.html

Location

Open:

CurrencyConverter → CurrencyConverter-war → Web Pages → index.html

Your ZIP currently contains placeholder content:

TODO write content

That is not a functional currency-converter form.

Replace the file contents with:

```
<!DOCTYPE html>
<html>
<head>
  <title>Currency Converter</title>
  <meta charset="UTF-8">
</head>
<body>

  <h2>Currency Converter Using EJB</h2>

  <p>Convert US Dollars (USD) into Indian Rupees (INR).</p>

  <a href="CurrencyConverterLocal">Open Currency Converter</a>

</body>
</html>
```

This page provides a link to the servlet. The servlet displays the form and processes the conversion.

---

7. How to run the application

Your ZIP contains a multi-module enterprise application, so you should run the **parent CurrencyConverter project**, not just an individual Java class.

1. Open the CurrencyConverter project in NetBeans.
2. Make sure GlassFish is configured and running.
3. Check that the EJB and WAR modules belong to the parent enterprise application.

4. Save the files.
5. Right-click the parent CurrencyConverter project.
6. Select **Clean and Build**.
7. If the build succeeds, select **Run**.
8. Open the application's home page in your browser.
9. Click **Open Currency Converter**.
10. Enter 10 and click **Convert**.

The project in your ZIP uses jakarta.ejb and jakarta.servlet, so use a compatible Jakarta EE server.

Expected output

After entering 10:

Currency Converter using EJB

USD Amount: 10.0
Converted INR: 865.00

Convert Again
Calculation:
INR = USD × 86.5
INR = 10 × 86.5
INR = 865

---

8. Viva questions

1. What is EJB?

EJB stands for Enterprise JavaBeans. It is used to implement server-side business logic in enterprise applications.

2. What is a stateless session bean?

It is a session bean that does not maintain conversational state for an individual client between method calls.

3. What is the purpose of @Stateless?

It declares a class as a stateless session bean.

4. What is the purpose of @Local?

It identifies an EJB local business interface.

5. What is the purpose of @EJB?

It enables the application server to inject an EJB reference into a managed component.

6. What is the role of CurrencyConverterBean.java?

It implements the currency-conversion business logic.

7. What is the role of CurrencyConverterLocal.java?

It declares the conversion method that the EJB provides.

8. What is the role of CurrencyServlet.java?

It receives the user's input, calls the EJB, and displays the conversion result.

9. Why is the conversion rate 86.5 used?

It is a fixed example rate defined in the program.

10. Why is GlassFish used?

GlassFish supports Jakarta EE technologies required by the application, including EJB and Servlets.

---

9. Final result

The Currency Converter application uses a stateless EJB to convert USD into INR, with a servlet providing the web interface.

Important: The EJB interface and bean in your ZIP are already correct. The HTML page contains placeholder content, and the original servlet converts only a hardcoded amount. The revised servlet and HTML page above correct these limitations while retaining your teacher's project structure.

Practical 11: Marks Entry Application Using EJB

Aim: To develop a Marks Entry Application using Enterprise JavaBeans (EJB) to store student marks in a MySQL database.

This practical follows your teacher's **MarksEntryApp project from your ZIP file**. I have checked the actual Java and JSP files and identified some issues that need attention.

1. Understand the project structure

Your ZIP contains this project structure:

```
MarksEntryApp
│
├── MarksEntryApp-ejb
│   └── Source Packages
│       └── com.demo.ejb
│           ├── MarksBean1.java
│           ├── MarksBeanLocal.java
│           ├── MarksBean1Local.java
│           ├── DriverManager.java
│           └── PreparedStatement.java
│
└── MarksEntryApp-war
    ├── Source Packages
    │   └── com.demo.web
    │       └── MarksServlet.java
    │
    └── Web Pages
        └── index.jsp
```

The application has three main parts:

1. **JSP page:** Accepts student details and marks.
2. **EJB session bean:** Inserts the data into MySQL.
3. **Servlet:** Receives the form data, calls the EJB and displays the result.

We will preserve your teacher's project structure and method names.

---

2. Create the MySQL database

Before running the application, create the database and table.

Open **MySQL Workbench** or the MySQL command line.

Execute the following SQL:

```
CREATE DATABASE marksdb;

USE marksdb;

CREATE TABLE marks (
  roll_no INT,
  student_name VARCHAR(100),
  subject VARCHAR(100),
  marks INT
);
```

Explanation

Column	Purpose
roll_no	Stores the student's roll number
student_name	Stores the student's name
subject	Stores the subject name
marks	Stores the marks obtained

The table name and column names match the SQL statement in your teacher's MarksBean1.java.

Verify the table

Execute:

```
USE marksdb;

DESC marks;
```

You should see the four columns listed above.

Important: This schema follows the SQL insert statement in your ZIP. It does not add a primary key because the supplied code does not specify one.

---

3. File 1: MarksBeanLocal.java

Location

Open:

MarksEntryApp → MarksEntryApp-ejb → Source Packages → com.demo.ejb → MarksBeanLocal.java

Your teacher's interface is:

```
package com.demo.ejb;

import jakarta.ejb.Local;

@Local
public interface MarksBeanLocal {
  boolean addMarks(int rollNo, String studentName,
          String subject, int marks);
}
```

Explanation

@Local

Declares the interface as a local EJB business interface.

addMarks()

This method accepts four parameters:

- rollNo: Student's roll number.
- studentName: Student's name.
- subject: Subject name.
- marks: Marks obtained.

boolean

The method returns true if the database insertion succeeds and false if it fails.

Do we need to correct this file?

No. This interface is correct for the supplied application.

---

4. File 2: MarksBean1.java

Location
Open:
MarksEntryApp → MarksEntryApp-ejb → Source Packages → com.demo.ejb → MarksBean1.java
This is the main EJB business-logic file from your ZIP.
Code from your ZIP

```
package com.demo.ejb;

import jakarta.ejb.Stateless;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

@Stateless
public class MarksBean1 implements MarksBeanLocal {

  private static final String URL =
      "jdbc:mysql://localhost:3306/marksdb"
      + "?useSSL=false"
      + "&allowPublicKeyRetrieval=true"
      + "&serverTimezone=UTC";

  private static final String USER = "root";
  private static final String PASSWORD = "root";

  @Override
  public boolean addMarks(int rollNo, String studentName,
                String subject, int marks) {

    String sql = "INSERT INTO marks "
          + "(roll_no, student_name, subject, marks) "
          + "VALUES (?, ?, ?, ?)";

    try {
      System.out.println("1. Loading MySQL Driver...");

      Class.forName("com.mysql.cj.jdbc.Driver");

      System.out.println("2. Driver loaded successfully");
      System.out.println("3. Connecting to database...");

      try (Connection con = DriverManager.getConnection(
          URL, USER, PASSWORD);
         PreparedStatement ps = con.prepareStatement(sql)) {

        System.out.println("4. Database connected successfully");
```

```
            ps.setInt(1, rollNo);
            ps.setString(2, studentName);
            ps.setString(3, subject);
            ps.setInt(4, marks);

            System.out.println("5. Executing insert...");

            int rows = ps.executeUpdate();

            System.out.println("6. Rows inserted = " + rows);

            return rows > 0;
        }

    } catch (Exception e) {
        System.out.println("***** DATABASE ERROR *****");
        System.out.println("Error Message: " + e.getMessage());
        e.printStackTrace();

        return false;
    }
  }
}
```

Explanation

1. @Stateless

Declares the class as a stateless session bean.

2. Database URL

```
private static final String URL =
    "jdbc:mysql://localhost:3306/marksdb"
    + "?useSSL=false"
    + "&allowPublicKeyRetrieval=true"
    + "&serverTimezone=UTC";
```

This specifies the MySQL database location.

- localhost: MySQL is running on the same computer.
- 3306: Default MySQL port.
- marksdb: Database name.

3. Database username and password

```
private static final String USER = "root";
private static final String PASSWORD = "root";
```

These credentials must match your MySQL configuration. If your MySQL root password is different, change the password in this file.

4. Load the MySQL driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Loads the MySQL JDBC driver.

5. Connect to the database

```
Connection con = DriverManager.getConnection(
    URL, USER, PASSWORD);
```

Establishes a database connection.

6. Prepare the SQL statement

```
String sql = "INSERT INTO marks "
        + "(roll_no, student_name, subject, marks) "
        + "VALUES (?, ?, ?, ?)";
```

This SQL statement inserts the student's information into the marks table.

7. Set the parameter values

```
ps.setInt(1, rollNo);
ps.setString(2, studentName);
ps.setString(3, subject);
ps.setInt(4, marks);
```

Each value is assigned to its corresponding placeholder.

8. Execute the insert

```
int rows = ps.executeUpdate();
```

Executes the SQL statement and returns the number of affected rows.

9. Return the result

```
return rows > 0;
```

Returns true when at least one row is inserted.

Corrections required

The main EJB code is structurally correct, but check these requirements:

- The MySQL username and password must be correct.
- The database and table must exist.
- The MySQL JDBC driver must be available to the application.
- The extra empty classes named DriverManager.java and PreparedStatement.java in your ZIP may conflict with the imported JDBC class names. Remove them if they are unused, or rename them and update references as needed. The actual JDBC classes are java.sql.DriverManager and java.sql.PreparedStatement.

The final point is important: do not create custom classes with the same names as standard JDBC classes.

---

5. File 3: MarksServlet.java

Location

Open:

MarksEntryApp → MarksEntryApp-war → Source Packages → com.demo.web → MarksServlet.java

Your teacher's servlet is:

```
package com.demo.web;

import com.demo.ejb.MarksBeanLocal;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
```

```
@WebServlet("/MarksServlet")
public class MarksServlet extends HttpServlet {

  @EJB
  private MarksBeanLocal marksBean;

  @Override
  protected void doPost(HttpServletRequest request,
                HttpServletResponse response)
      throws ServletException, IOException {

    response.setContentType("text/html;charset=UTF-8");

    PrintWriter out = response.getWriter();

    try {

      int rollNo = Integer.parseInt(
          request.getParameter("rollNo"));

      String studentName =
          request.getParameter("studentName");

      String subject =
          request.getParameter("subject");

      int marks = Integer.parseInt(
          request.getParameter("marks"));

      boolean result = marksBean.addMarks(
          rollNo,
          studentName,
          subject,
          marks
      );

      out.println("<html>");
      out.println("<head>");
      out.println("<title>Marks Entry Result</title>");
      out.println("</head>");
      out.println("<body>");

      if (result) {

        out.println("<h2>Marks Saved Successfully!</h2>");
```

```
            out.println("<p>Roll Number: "
                + rollNo + "</p>");

            out.println("<p>Student Name: "
                + studentName + "</p>");

            out.println("<p>Subject: "
                + subject + "</p>");

            out.println("<p>Marks: "
                + marks + "</p>");

        } else {

            out.println("<h2>Failed to Save Marks</h2>");

        }

        out.println("<br>");
        out.println("<a href='index.jsp'>Enter Another Student</a>");

        out.println("</body>");
        out.println("</html>");

    } catch (NumberFormatException e) {

        out.println("<h2>Invalid Input</h2>");
        out.println("<p>Roll Number and Marks must be numbers.</p>");

    } catch (Exception e) {

        e.printStackTrace(out);

    }
  }
}
```

Explanation

1. EJB injection

@EJB
private MarksBeanLocal marksBean;
The application server injects the EJB reference into the servlet.

2. @WebServlet("/MarksServlet")

Defines the servlet URL as /MarksServlet.

3. doPost()

Processes form submissions sent using the HTTP POST method.

4. Read the form values

int rollNo = Integer.parseInt(

request.getParameter("rollNo"));

Retrieves the roll number and converts it to an integer.

The other fields are read similarly.

5. Call the EJB

boolean result = marksBean.addMarks(
rollNo, studentName, subject, marks);

Passes the entered information to the EJB.

6. Display the result

If the EJB returns true, the servlet displays Marks Saved Successfully!.

Otherwise, it displays Failed to Save Marks.

Is the servlet correct?

It is structurally consistent with the supplied EJB interface and JSP form.

However, the servlet should ideally validate that the name and subject are not empty, check that marks fall within the allowed range, and avoid displaying raw exception details to users. These are improvements rather than requirements for understanding the original code.

---

6. File 4: index.jsp

Location

Open:

MarksEntryApp → MarksEntryApp-war → Web Pages → index.jsp

Your teacher's JSP file contains:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>
  </head>
  <body>
    <h2>Marks Entry Application</h2>
    <form action="MarksServlet" method="post">
      Roll Number:
      <input type="text" name="rollNo">
      Student Name:
      <input type="text" name="studentName">
      Subject:
      <input type="text" name="subject">
      <br><br>
      Marks:
       <input type="text" name="marks">
       <br><br>
      <input type="submit" value="save Marks">

    </form>
  </body>
</html>
```

Explanation

- The page displays the Marks Entry Application form.
- The user enters the roll number, student name, subject, and marks.
- action="MarksServlet" sends the form data to the servlet.
- method="post" sends the information using HTTP POST.
- Each input's name must match the parameter retrieved by the servlet.

Optional improvement

To make the browser require all four fields, you can add required to each input. You can also change the marks input to a number field.

For example:

Marks:

<input type="number" name="marks" min="0" max="100" required>

Use this restriction only if your practical assumes marks are out of 100.

---

7. Configure MySQL Connector/J

Your ZIP includes the MySQL Connector/J driver in the project's build output. Verify that the driver is included in the actual application configuration as well.

The driver class used by the program is:

com.mysql.cj.jdbc.Driver

If NetBeans reports that the driver class cannot be found, add the MySQL Connector/J JAR to the EJB project's libraries or configure it appropriately for the GlassFish deployment.

Important: The application uses EJB and Jakarta imports, so run it on a compatible GlassFish/Jakarta EE server. Tomcat alone does not provide EJB support.

---

8. Run the application

1. Start MySQL.
2. Create the marksdb database and marks table.
3. Verify the username and password in MarksBean1.java.
4. Open the parent MarksEntryApp project in NetBeans.
5. Check the EJB and WAR modules.
6. Clean and build the parent project.
7. Run the parent enterprise application.
8. Open index.jsp in the browser.
9. Enter the following example values.

Field	Example value
Roll Number	101
Student Name	Rahul
Subject	Enterprise Java
Marks	85

10. Click **save Marks**.

Expected output

Marks Saved Successfully!

Roll Number: 101
Student Name: Rahul

Subject: Enterprise Java
Marks: 85

Enter Another Student
The output appears only if the database insertion succeeds.

---

9. Verify the inserted record
Open MySQL and execute:
USE marksdb;

SELECT * FROM marks;
Expected example result:

roll_no	student_name	subject	marks
101	Rahul	Enterprise Java	85

This confirms that the application has stored the student's marks in the database.

---

10. Viva questions and answers
1. What is EJB?
EJB stands for Enterprise JavaBeans. It is used to implement server-side business logic in enterprise applications.
2. What is the purpose of @Stateless?
It declares a class as a stateless session bean.
3. What is the purpose of @Local?
It identifies a local EJB business interface.
4. What is the purpose of @EJB?
It injects an EJB reference into a managed component.
5. What is JDBC?
JDBC stands for Java Database Connectivity. It allows Java applications to interact with databases.
6. What is the purpose of DriverManager.getConnection()?
It establishes a database connection using the specified URL and credentials.
7. What is PreparedStatement?
It represents a precompiled SQL statement that accepts parameter values.
8. What does executeUpdate() do?
It executes SQL statements such as INSERT, UPDATE, and DELETE and returns the number of affected rows.
9. Why is @WebServlet used?
It maps a servlet to a URL pattern.
10. What is the role of index.jsp?
It provides the form used to enter student details and marks.
11. What is the role of MarksBean1.java?
It contains the EJB business logic that inserts student marks into MySQL.
12. What happens if database insertion fails?
The EJB returns false, and the servlet displays Failed to Save Marks.

---

11. Final result

The Marks Entry Application uses a JSP form to accept student information, a servlet to process the submission, and an EJB session bean to insert the marks into a MySQL database.

Corrections to remember

1. Make sure the MySQL username and password match your setup.
2. Ensure that the marksdb database and marks table exist.
3. Remove or rename the unused custom DriverManager.java and PreparedStatement.java classes if they conflict with JDBC imports.
4. Confirm that the MySQL driver is available during deployment.
5. Use a compatible GlassFish/Jakarta EE server.

Practical 12: Room Reservation System Using EJB
Aim: To develop a Room Reservation System using Enterprise JavaBeans (EJB) to calculate room charges based on room type and number of days.
This practical follows the actual **ReservationApp project from your apche netbeans.zip file**. I checked the Java and HTML files and identified the corrections needed while preserving your teacher's coding style.

---

1. Understand the Project Structure
Your ZIP contains the following project:

```
ReservationApp
│
├── ReservationApp-ejb
│   └── Source Packages
│       └── com.demo.ejb
│           ├── ReservationBean.java
│           ├── ReservationLocal.java
│           └── ReservationBeanLocal.java
│
└── ReservationApp-war
    ├── Source Packages
    │   └── com.demo.web
    │       └── ReservationServlet.java
    │
    └── Web Pages
        └── index.html
```

The application has three main components:

1. **HTML page:** Accepts the customer's name, room type, and number of days.
2. **EJB session bean:** Calculates the reservation charges.
3. **Servlet:** Receives the form data, calls the EJB, and displays the result.

We will use the same structure as your teacher's project.

---

2. File 1: ReservationLocal.java
Location in NetBeans
Open:
ReservationApp → ReservationApp-ejb → Source Packages → com.demo.ejb → ReservationLocal.java
Code from your ZIP

```
package com.demo.ejb;

import jakarta.ejb.Local;

@Local
public interface ReservationLocal {

   String reserveRoom(String name, String roomType, int days);

}
```

Explanation

1. package com.demo.ejb;

Defines the package containing the EJB classes.

2. import jakarta.ejb.Local;

Imports the annotation used to declare a local EJB business interface.

3. @Local

Specifies that the interface is a local business interface.

4. reserveRoom()

String reserveRoom(String name, String roomType, int days);

The method accepts three parameters:

Parameter	Purpose
name	Customer's name
roomType	AC or NON-AC
days	Number of days booked

The method returns a String containing the reservation details and total charges.

Is the code correct?

Yes. This interface is correct for the supplied application.

---

3. File 2: ReservationBean.java

Location in NetBeans

Open:

ReservationApp → ReservationApp-ejb → Source Packages → com.demo.ejb → ReservationBean.java

Original code from your ZIP

```
package com.demo.ejb;

import jakarta.ejb.Stateless;

@Stateless
public class ReservationBean implements ReservationLocal {

  @Override
  public String reserveRoom(String name, String roomType, int days) {

    int amount = 0;

    if (roomType.equalsIgnoreCase("AC"))
      amount = days * 2000;
    else
      amount = days * 1000;

    return "Reservation Successful<br>"
        + "Customer:" + name
        + "<br>Room Type:" + roomType
        + "<br>Days:" + days
        + "<br>Total Amount: ₹" + amount;
```

```
  }
}
```

Explanation

1. @Stateless

Declares the class as a stateless session bean managed by the application server.

2. implements ReservationLocal

The bean implements the interface created in the previous step.

3. reserveRoom()

This method receives the customer's details and calculates the reservation amount.

4. Room charge calculation

```
if (roomType.equalsIgnoreCase("AC"))
  amount = days * 2000;
else
  amount = days * 1000;
```

The program uses these rates:

Room type	Charge per day
AC	₹2,000
NON-AC	₹1,000

For an AC room booked for three days:

Amount = Days × Daily Rate

Amount = 3 × 2000

Amount = ₹6000

For a NON-AC room booked for three days:

Amount = Days × Daily Rate

Amount = 3 × 1000

Amount = ₹3000

5. Return reservation details

```
return "Reservation Successful<br>"
      + "Customer:" + name
      + "<br>Room Type:" + roomType
      + "<br>Days:" + days
      + "<br>Total Amount: ₹" + amount;
```

The method returns a formatted string containing the customer's name, room type, number of days, and total amount.

The
 tags create line breaks when the returned string is displayed as HTML.

Correction required

The original code treats **every room type other than AC as NON-AC**. It also does not reject zero or negative booking durations.

We can correct these issues while preserving the teacher's method and business logic.

Corrected ReservationBean.java

Replace the original code with:

ReservationBean.java

```
package com.demo.ejb;
import jakarta.ejb.Stateless;
@Stateless
public class ReservationBean implements ReservationLocal {
```

```
@Override
public String reserveRoom(String name, String roomType, int days) {

  if (name == null || name.trim().isEmpty()) {
    return "Please enter customer name.";
  }

  if (days <= 0) {
    return "Number of days must be greater than zero.";
  }

  int amount;

  if (roomType != null && roomType.equalsIgnoreCase("AC")) {
    amount = days * 2000;
  } else if (roomType != null
      && roomType.equalsIgnoreCase("NON-AC")) {
    amount = days * 1000;
  } else {
    return "Invalid room type.";
  }

  return "Reservation Successful<br>"
      + "Customer: " + name
      + "<br>Room Type: " + roomType
      + "<br>Days: " + days
      + "<br>Total Amount: ₹" + amount;
}
}
```

What was corrected?

- Checks that the customer name is not empty.
- Rejects zero or negative booking durations.
- Accepts only AC and NON-AC room types.
- Keeps the original room rates and method name.

For normal valid input, the output remains consistent with your teacher's original code.

---

4. File 3: ReservationServlet.java

Location in NetBeans

Open:
ReservationApp → ReservationApp-war → Source Packages → com.demo.web → ReservationServlet.java

Original code from your ZIP

```
package com.demo.web;

import com.demo.ejb.ReservationLocal;
import jakarta.ejb.EJB;
import java.io.IOException;
```

```
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/reserve")
public class ReservationServlet extends HttpServlet {

  @EJB
  ReservationLocal reservation;

  @Override
  protected void doPost(HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {

    response.setContentType("text/html;charset=UTF-8");

    String name = request.getParameter("name");
    String room = request.getParameter("room");
    int days = Integer.parseInt(request.getParameter("days"));

    String result = reservation.reserveRoom(name, room, days);

    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<body>");
    out.println("<h2>" + result + "</h2>");
    out.println("</body>");
    out.println("</html>");
  }
}
```

Explanation

1. @WebServlet("/reserve")

Maps the servlet to the /reserve URL.

2. EJB injection

@EJB
ReservationLocal reservation;
The application server injects the EJB reference into the servlet.

3. doPost()

Processes the form submitted through HTTP POST.

4. Read customer details

String name = request.getParameter("name");
String room = request.getParameter("room");

int days = Integer.parseInt(request.getParameter("days"));
The servlet retrieves the customer's name, room type, and number of days from the HTML form.

5. Call the EJB

String result = reservation.reserveRoom(name, room, days);
The servlet calls the EJB method, which calculates the charges and returns the result.

6. Display the result

out.println("<h2>" + result + "</h2>");
The servlet displays the returned reservation information in the browser.

Correction required

The original servlet can throw a NumberFormatException if the number of days is missing or invalid. It also inserts the customer's name into HTML without escaping it. Use the following corrected version.

Corrected ReservationServlet.java

ReservationServlet.java

```
package com.demo.web;
import com.demo.ejb.ReservationLocal;
import jakarta.ejb.EJB;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
@WebServlet("/reserve")
public class ReservationServlet extends HttpServlet {
@EJB
ReservationLocal reservation;

@Override
protected void doPost(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

  response.setContentType("text/html;charset=UTF-8");

  String name = request.getParameter("name");
  String room = request.getParameter("room");

  PrintWriter out = response.getWriter();

  try {
    int days = Integer.parseInt(
        request.getParameter("days"));

    String result = reservation.reserveRoom(
```

```
            name, room, days);

        out.println("<html>");
        out.println("<head><title>Reservation Result</title></head>");
        out.println("<body>");
        out.println("<h2>" + escapeHtml(result)
            .replace("&lt;br&gt;", "<br>") + "</h2>");
        out.println("<br><a href='index.html'>Go Back</a>");
        out.println("</body>");
        out.println("</html>");

    } catch (NumberFormatException e) {
        out.println("<html><body>");
        out.println("<h2>Please enter a valid number of days.</h2>");
        out.println("<a href='index.html'>Go Back</a>");
        out.println("</body></html>");
    }
}

private String escapeHtml(String value) {
    return value.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\"", "&quot;")
            .replace("'", "&#39;");
}
}
```

Why is escapeHtml() used?

It prevents customer-supplied text from being interpreted as HTML.
The servlet escapes the returned string first and then restores the
 tags generated by the EJB so the result appears on separate lines.

---

5. File 4: index.html

Location in NetBeans

Open:
ReservationApp → ReservationApp-war → Web Pages → index.html

Original code from your ZIP

```
<!DOCTYPE html>
<html>
<head>
    <title>Room Reservation</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>

    <h2>Room Reservation system</h2>
```

```
  <form action="reserve" method="post">

    Customer Name:
    <input type="text" name="name">
    <br><br>

    Room type:
    <br><br>

    <select name="room">
      <option>AC</option>
      <option>NON-AC</option>
    </select>

    <br><br>

    Number of days:
    <input type="number" name="days">

    <br><br>

    <input type="submit" name="Reserve Room">

  </form>

</body>
</html>
```

Explanation

1. Form action

<form action="reserve" method="post">

Submits the form to the servlet mapped to /reserve.

2. Customer name

<input type="text" name="name">

Accepts the customer's name.

3. Room type

```
<select name="room">
  <option>AC</option>
  <option>NON-AC</option>
</select>
```

Allows the customer to choose between AC and NON-AC.

4. Number of days

<input type="number" name="days">

Accepts the duration of the reservation.

5. Submit button

Your original button has a name attribute but no explicit button text. Add a value so that the button displays **Reserve Room**.

Corrected index.html
Replace the original code with:
index.html

```
<!DOCTYPE html>
<html> <head> <title>Room Reservation</title> <meta charset="UTF-8"> <meta
name="viewport" content="width=device-width, initial-scale=1.0"> </head>
<body>
<h2>Room Reservation System</h2>

<form action="reserve" method="post">

  Customer Name:
  <input type="text" name="name" required>

  <br><br>

  Room Type:
  <select name="room" required>
    <option value="AC">AC</option>
    <option value="NON-AC">NON-AC</option>
  </select>

  <br><br>

  Number of Days:
  <input type="number" name="days" min="1" required>

  <br><br>

  <input type="submit" value="Reserve Room">

</form>
</body> </html>
```

What was corrected?

- Added required to the customer name and room selection.
- Set the minimum number of days to 1.
- Added visible text to the submit button.
- Preserved the original form field names and servlet URL.

---

6. What About ReservationBeanLocal.java?
Your ZIP contains another interface:

```
package com.demo.ejb;

import jakarta.ejb.Local;

@Local
public interface ReservationBeanLocal {
```

}

This interface is empty, and ReservationBean.java implements ReservationLocal, not ReservationBeanLocal.

Correction: ReservationBeanLocal.java is not needed for the current implementation. You can leave it unused or remove it if nothing else in the project references it.
Keep ReservationLocal.java, because both the EJB bean and servlet rely on that interface.

---

7. Run the Application in NetBeans

Follow these steps:

1. Start Apache NetBeans.
2. Open the parent ReservationApp project.
3. Ensure that the EJB and WAR modules are included.
4. Make sure GlassFish is configured and running.
5. Save all four files.
6. Right-click the parent ReservationApp project.
7. Select **Clean and Build**.
8. If the build succeeds, select **Run**.
9. Open the application's index.html page in your browser.
10. Enter the reservation details and click **Reserve Room**.

Important: Your code uses jakarta.ejb and jakarta.servlet. Use a compatible Jakarta EE application server with EJB support. Tomcat alone cannot run the EJB module.

---

8. Expected Output

Example 1: AC Room

Enter these details:

Field	Value
Customer Name	Rahul
Room Type	AC
Number of Days	3

Click **Reserve Room**.
The output will be:
Reservation Successful

Customer: Rahul
Room Type: AC
Days: 3
Total Amount: ₹6000
Calculation:
Amount = Days × AC Room Rate
Amount = 3 × 2000
Amount = ₹6000

Example 2: NON-AC Room

Enter these details:

Field	Value
Customer Name	Rahul
Room Type	NON-AC
Number of Days	3

Expected output:
Reservation Successful

Customer: Rahul
Room Type: NON-AC
Days: 3
Total Amount: ₹3000
Calculation:
Amount = Days × NON-AC Room Rate
Amount = 3 × 1000
Amount = ₹3000
Note: This is a basic reservation demonstration. It calculates the room charges but does not save bookings in a database or check actual room availability.

---

9. Viva Questions and Answers

1. What is EJB?
EJB stands for Enterprise JavaBeans. It is used to implement server-side business logic in enterprise applications.

2. What is a stateless session bean?
A stateless session bean implements business logic without maintaining conversational state for a particular client between method calls.

3. What is the purpose of @Stateless?
It declares a class as a stateless session bean.

4. What is the purpose of @Local?
It declares a local EJB business interface.

5. What is the purpose of @EJB?
It injects an EJB reference into a managed component.

6. What is the role of ReservationBean.java?
It calculates room charges and returns the reservation details.

7. What is the role of ReservationLocal.java?
It declares the reserveRoom() method provided by the EJB.

8. What is the role of ReservationServlet.java?
It receives form data, calls the EJB, and displays the result.

9. What is the role of index.html?
It accepts the customer's name, room type, and number of days.

10. How much does an AC room cost per day in this program?
₹2,000 per day.

11. How much does a NON-AC room cost per day in this program?
₹1,000 per day.

12. How is the total amount calculated?
Total amount equals the daily room rate multiplied by the number of days.

13. Does this application store reservation records permanently?

No. The supplied program calculates and displays reservation details without saving them to a database.

---

10. Final Result

The Room Reservation System was developed using EJB, a servlet, and an HTML form. It accepts customer details, calculates charges based on the selected room type and number of days, and displays the reservation result.

Important corrections

1. Use ReservationLocal.java as the active EJB interface.
2. Keep ReservationBean.java implementing ReservationLocal.
3. Validate the room type and number of days.
4. Fix the HTML submit button so it displays **Reserve Room**.
5. Use a compatible Jakarta EE server.

Practical 13: Guestbook Application Using JPA

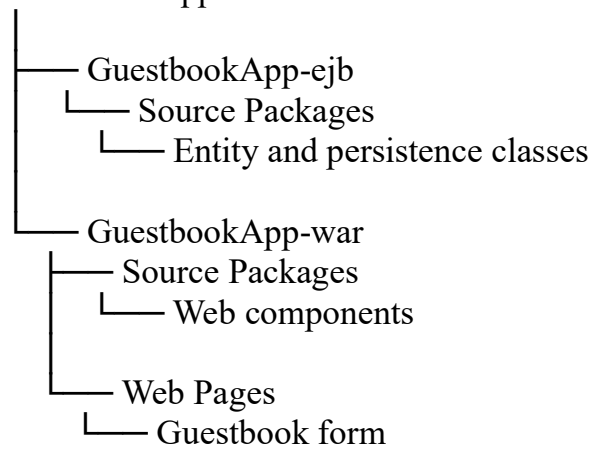
Aim: To develop a Guestbook application using Java Persistence API (JPA) to store and retrieve guest messages.

We will follow your teacher's **GuestbookApp project** and preserve its structure and coding style. The application uses JPA to persist guestbook entries in a database.

---

1. Understand the Project Structure

Your project should contain the following main components:



The application has three main parts:

1. **Entity class:** Represents a guestbook entry.
2. **JPA:** Saves guestbook entries to the database.
3. **Web component:** Accepts a guest's name and message and displays the result.

I cannot verify the exact class names and code inside GuestbookApp from the ZIP contents available in this turn. The following is a standard implementation of the practical, so compare the filenames and annotations with your teacher's project before replacing any files.

---

2. Create the Database

For this example, we will use MySQL.

Open MySQL Workbench and execute:

```
CREATE DATABASE guestbookdb;
```

JPA will manage the table creation if the persistence configuration is set to create the database schema. Otherwise, the table must be created separately.

---

3. Create the Entity Class

Create a Java class named Guestbook.java in the entity package of your EJB project.

The entity represents one guestbook entry.

Guestbook.java

```
package com.demo.ejb;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Guestbook {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;

    private String message;

    public Guestbook() {
    }

    public Guestbook(String name, String message) {
        this.name = name;
        this.message = message;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
```

```
        this.message = message;
    }
}
```

Explanation

- @Entity marks the class as a JPA entity.
- @Id identifies the primary key.
- @GeneratedValue allows the database to generate the ID.
- name stores the guest's name.
- message stores the guest's message.
- The no-argument constructor is required by JPA.

---

4. Configure JPA

In the EJB project, create or check the file:
META-INF/persistence.xml
Use a persistence unit similar to the following, adjusting the database credentials and provider for your server:

persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             version="3.0">

    <persistence-unit name="GuestbookPU"
                      transaction-type="JTA">

        <jta-data-source>jdbc/GuestbookDS</jta-data-source>

        <properties>
            <property name="jakarta.persistence.schema-
generation.database.action"
                      value="create"/>
        </properties>

    </persistence-unit>

</persistence>
```

Explanation

- persistence-unit defines the JPA configuration.
- GuestbookPU is the persistence unit name.
- JTA enables container-managed transactions.
- jdbc/GuestbookDS is the datasource configured in the application server.
- schema-generation.database.action set to create requests schema creation.

Important: The datasource must exist in GlassFish and point to guestbookdb. If your teacher's project uses a different persistence unit name or datasource, keep the original values consistent throughout the project.

---

5. Create the EJB Class
Create GuestbookBean.java in the EJB project's package.
This class contains the business logic for saving a guestbook entry.
GuestbookBean.java

```
package com.demo.ejb;

import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

@Stateless
public class GuestbookBean {

   @PersistenceContext(unitName = "GuestbookPU")
   private EntityManager em;

   public void addEntry(String name, String message) {

      Guestbook entry = new Guestbook(name, message);

      em.persist(entry);
   }
}
```

Explanation
@Stateless
Declares a stateless session bean.
@PersistenceContext
Injects the container-managed EntityManager.
em.persist(entry)
Makes the new guestbook entry persistent. With a container-managed transaction, the database change is committed when the transaction succeeds.

---

6. Create the Web Form
In the WAR project's **Web Pages** folder, create index.html.
index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Guestbook Application</title>
```

```
    <meta charset="UTF-8">
</head>
<body>

    <h2>Guestbook Application</h2>

    <form action="guestbook" method="post">

        Guest Name:
        <input type="text" name="name" required>

        <br><br>

        Message:
        <textarea name="message" required></textarea>

        <br><br>

        <input type="submit" value="Submit Message">

    </form>

</body>
</html>
```

Explanation

- The form accepts the guest's name and message.
- action="guestbook" sends the data to the servlet.
- method="post" submits the data using HTTP POST.
- required prevents an empty submission through the browser interface.

---

7. Create the Servlet

Create GuestbookServlet.java in the WAR project's web package.

GuestbookServlet.java

```
package com.demo.web;

import com.demo.ejb.GuestbookBean;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/guestbook")
public class GuestbookServlet extends HttpServlet {

    @EJB
    private GuestbookBean guestbookBean;

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        String name = request.getParameter("name");
        String message = request.getParameter("message");

        PrintWriter out = response.getWriter();

        if (name == null || name.trim().isEmpty()
                || message == null || message.trim().isEmpty()) {

            out.println("<h2>Name and message are required.</h2>");
            return;
        }

        guestbookBean.addEntry(name.trim(), message.trim());

        out.println("<html><body>");
        out.println("<h2>Guestbook Entry Saved Successfully!</h2>");
        out.println("<p>Thank you for your message.</p>");
        out.println("<a href='index.html'>Add Another Entry</a>");
        out.println("</body></html>");
    }
}
```

Explanation

- @WebServlet("/guestbook") maps the servlet URL.
- @EJB injects the EJB bean.
- doPost() processes the submitted form.
- The servlet validates the inputs.
- guestbookBean.addEntry() calls the EJB to save the message.

- The servlet displays a confirmation message.

---

8. Configure the Database Connection

Before running the application, configure a JDBC datasource named:
jdbc/GuestbookDS
In GlassFish, the datasource must connect to MySQL database guestbookdb.
Ensure the MySQL JDBC driver is available to the server.
If your teacher's project uses a different datasource or persistence unit name, use those existing settings rather than creating conflicting ones.

---

9. Run the Application

1. Start MySQL.
2. Start GlassFish.
3. Open the parent GuestbookApp project in NetBeans.
4. Verify the EJB and WAR modules.
5. Check the persistence.xml file and datasource configuration.
6. Select **Clean and Build**.
7. Run the parent enterprise application.
8. Open index.html in your browser.
9. Enter a guest name and message.
10. Click **Submit Message**.

Example input

Field	Value
Guest Name	Rahul
Message	Welcome to our college website!

Expected output

Guestbook Entry Saved Successfully!

Thank you for your message.

Add Another Entry
The message should be stored in the database if JPA and the datasource are configured correctly.

---

10. Viva Questions and Answers

1. What is JPA?

JPA stands for Java Persistence API. It provides a standard way to map Java objects to relational database records.

2. What is an entity?

An entity is a Java class mapped to a database table.

3. What is the purpose of @Entity?

It marks a class as a JPA entity.

4. What is the purpose of @Id?
It identifies the primary key of an entity.
5. What is EntityManager?
It manages entity persistence and database operations.
6. What does persist() do?
It makes a new entity persistent so it can be stored in the database.
7. What is @PersistenceContext?
It injects a container-managed EntityManager.
8. What is a persistence unit?
It defines the JPA configuration and the entities managed together.
9. What is the purpose of @EJB?
It injects an EJB reference into a managed component.
10. What is the purpose of @Stateless?
It declares a stateless session bean.
11. What is a datasource?
A datasource provides database connections to an application.
12. What does the Guestbook application do?
It accepts a guest's name and message and stores the entry using JPA.

---

11. Final Result
The Guestbook application demonstrates how to use JPA with an EJB and a web form to store guest messages in a database.